Instructor Guide

AI-Assisted Game Development — the single-day program, minute by minute

Prepared for Panteon Games — Codeo · One trainer · 09:30 – 17:30

This is the trainer's complete runbook. For every session it gives you the talk track per slide, the demo scripts with exact commands and expected output, every participant prompt verbatim, what to patrol for while they work, what a correct result looks like (taken from the canonical solution branches), and how to recover when things break. Participant-facing detail lives in the two lab guides and the exercise files in the repos (`coin-rush-workshop/Assets/Workshop/Ex0N_*/README.md`, `gem-hunter-workshop/Workshop/Ex0N_*.md`); this guide tells you how to *run* them.

How to read it: SAY a line to deliver · RUN a command or prompt, with what you should see · LOOK FOR what correct looks like · PATROL the wrong turns to catch while circulating. All CLI commands in Gem Hunter take `--project-path .` from the repo root; in Coin Rush the connected editor is implied.

PART 0 · BEFORE THE DAY

T-1 day Machines and accounts

- ☐ Every participant machine: both repos cloned, opened once in Unity 6 (so Library is built — first open takes minutes), `unity status` shows `ready`, `claude` starts and reads the project CLAUDE.md.
- ☐ Engineers: Claude Code access confirmed. Art: Scenario account with the studio style model visible; Meshy is not needed in this program.
- ☐ In gem-hunter-workshop on each machine: `./catchup.sh list` prints `complete ex01-start … ex09-start`. If a machine cloned without branches, `git fetch --all`.
- ☐ Team leads identified (engineering, art, production) — they receive the metrics template in the opening.

T-1 day Your own machine

- ☐ Two Unity instances: coin-rush-workshop (open `Assets/Workshop/Ex01_SceneSetup/Ex01_Start.unity`) and gem-hunter-workshop on `main`.
- ☐ **A second clone of gem-hunter-workshop at `ex06-start`** — your BUG-312 demo copy. Never demo the bug from your main clone; the fix must not leak into the branch participants sync to.
- ☐ Rehearse the BUG-312 demo end to end once (Part 3). Record your screen while you do — that recording is your fallback.
- ☐ Pre-generate a 7-gem fallback set from the studio style model (1024×1024, transparent background, one file per gem) for anyone whose Scenario access fails.
- ☐ Print: baseline metrics template ×3, art approval checklist ×(art headcount). Deck files open in order 01–06.

T-1 hour In the room

- ☐ Projector shows the terminal and the Unity editor side by side — the whole morning depends on the room seeing the editor react to the terminal.
- ☐ Smoke test on your machine: `unity command eval 'return 1+1;'` against both editors returns `2`.
- ☐ Terminal font at 18pt+; Unity UI scaled up; Claude Code in a wide pane so plans are readable from the back.

THE DAY, MINUTE BY MINUTE

Time	Block	Mode
09:30 – 09:45	Opening talk (deck 01, slides 1–6)	You talk
09:45 – 09:55	Everyone plays Gem Hunter Level 1 · metrics template handed out	They play
10:00 – 10:15	Unity CLI: why, three layers, how the agent drives it (deck 02, slides 1–4)	You talk
10:15 – 10:20	Demo: connect, discover, create (Coin Rush Ex01)	You demo
10:20 – 10:30	Hands-on: ground + player from the terminal (Ex01)	They do
10:30 – 10:45	Hands-on: agent writes PlayerController in plan mode (Ex02)	They do
10:45 – 10:52	Demo: coin prefab + live counting (Ex03)	You demo
10:52 – 11:00	[CliCommand], security, maturity, up next (deck 02, slides 6–9)	You talk

11:00 – 11:15	Break	
11:15 – 11:45	Three techniques: archaeology, contract, checkpoints (deck 03, slides 1–4)	You talk
11:45 – 12:25	Live demo BUG-312 on the ex06-start clone (slide 5)	You demo
12:25 – 12:30	Questions	
12:30 – 13:30	Lunch	
13:30 – 13:40	Track A environment check (deck 04, slides 1–2)	They check
13:40 – 14:10	Ex01 Recon + Ex02 Contract	They do
14:10 – 14:30	Ex03 LevelData, Ex04 ComboCounter, Ex05 Amethyst	You demo
14:30 – 14:45	Ex06 BUG-312	They do
14:45 – 14:55	Ex07 level_lint	They do
14:55 – 15:00	Ex08 briefing · Ex09 build kicked off	You demo
15:00 – 15:15	Break (build keeps running)	
15:15 – 15:25	Track B talk (deck 05, slides 1–2, 5)	You talk
15:25 – 16:00	Flow 1: generate » clean up » import » swap	They do
16:00 – 16:40	Flow 2: lights » VFX » verify » approval package	They do
16:40 – 16:45	Pack up outputs for the closing	
16:45 – 17:05	Show & tell + the merge (deck 06, slides 1–2)	They show
17:05 – 17:25	Four traps, next 90 days (slides 3–4)	You talk
17:25 – 17:30	Collect metrics templates · thanks (slide 5)	

PART 1 · 09:30 OPENING (DECK 01)

Slide 1 Title

SAY "You already use Claude Code every day, so nothing today is a tool introduction. This morning you build a game scene from an empty project without touching the mouse; this afternoon you work inside 4,800 lines none of you wrote. One rule runs through both: no proof, no done."

Slide 2 Measurement — record the baseline

SAY "Four numbers, recorded today, compared in 90 days: T1 average time from idea to merged code, T2 from art request to approved asset, T3 from bug report to verified fix, and the share of generated content rejected at first approval." Hand the template to the three team leads now. SAY "You fill in current values during the day from memory and your trackers; I collect them at 17:25." LOOK FOR at least one number per row by the closing — rough is fine, blank is not.

Slide 3 Core principle — the approval gate

SAY "Tools do the producing, people make the final call. Three concrete gates: every generated image or model passes the art director's approval; no agent-authored change merges without review; and for everything generated we record tool, model version and prompt — provenance. Track B rehearses the first gate today; Track A lives inside the second; both keep the third."

Slide 4 The arc of the day

SAY "Morning, from scratch: the agent drives the editor through Unity CLI — eval, plan mode, prefabs. Afternoon, discovery and intervention: map a foreign codebase, extend it without breaking it. The proof rule doesn't change between the two."

Slide 5 Today's game

SAY "Gem Hunter Match is Unity's official match-3 sample: board logic, boosters, a shop, three levels, a 2D lighting and VFX showcase. About 4,800 lines in 34 scripts — small enough to grasp in a day, real enough to hurt. Engineering extends it and hunts a live bug; art reskins it in your style; at the closing both land in one build."

Slide 6 Schedule, then play

Walk the six rows in ten seconds each. Then: SAY "Open gem-hunter-workshop, press Play on Level 1, and finish or lose it once. Notice three things: the move counter, the goals at the top, and what a booster does when you swap it." LOOK FOR

everyone has actually seen the move counter tick — the BUG-312 demo depends on that mental model. Ten minutes, hard stop at 09:55.

PART 2 · 10:00 UNITY CLI FROM SCRATCH (DECK 02 + COIN RUSH)

Slides 1–4 Why · three layers · how the agent drives the editor (15 min)

SAY (slide 2) "Since July 2026 Unity CLI is the official way for agents to work with Unity — community MCP servers are replaced by a Unity-supported tool with version guarantees, and existing MCP setups migrate through a compatibility mode. For a live-ops studio the value is one number: time from bug report to verified fix." **SAY** (slide 3) "Three layers. The `unity` command itself: installs, projects, authentication, the same binary on CI. `com.unity.pipeline`: a package that lets scripts and agents drive a *running* editor — scene operations, imports, builds. And `eval`: live C# inside the editor, no compile loop. The pipeline package is still experimental; slide 8 has the criteria for calling it production-ready." **SAY** (slide 4) walk the five steps: task in → connect via pipeline → verify with eval → fix on a branch → human approves the merge. "Step five is not decoration. It is the whole point."

Aside Two halves of the CLI — with or without an editor (2 min, land it here)

SAY "The CLI has two halves and today lives almost entirely in one of them. Half one needs no editor open: `unity install`, `unity projects`, `unity auth`, `unity license`, and the CI trio `unity build`, `unity test`, `unity run` — those start their own headless editor in batch mode, do the job, exit with a code. Half two is a remote control for an editor that is already running with the pipeline package loaded: `unity status`, `unity command …` — eval, open_scene, attach_script, the live build and build_status, your own level_lint. No editor, no connection; a compile error on open puts the editor in Safe Mode and the connection fails the same way." **LOOK FOR** the moment people realise the morning's `unity status → ready` check is a hard prerequisite, not a formality. Plant the payoff: "this afternoon's iOS build runs through the live editor; the closing shows the same build the CI way, on a closed project."

Demo Connect, discover, create — Coin Rush Ex01 (5 min)

Everyone has `Ex01_Start.unity` open. You, on the projector, terminal beside the editor:

```
$ unity status
# → connected editor … state: ready
$ unity command
# → the list of commands this editor exposes (eval, build, build_status, console, editor_play, …)
$ unity command eval 'new UnityEngine.GameObject("Hello");'
```

LOOK FOR "Hello" appears in the Hierarchy the instant the command returns — point at it. Then delete it:

```
$ unity command eval 'UnityEngine.Object.DestroyImmediate(UnityEngine.GameObject.Find("Hello"));'
```

SAY "That is the whole trick. The terminal is now an editor window. Everything the agent does today goes through this door."

Hands-on Ex01 — ground and player from the terminal (10 min)

SAY "Your turn: the README in Ex01_SceneSetup has four commands. Run them one at a time and watch the Hierarchy. No mouse in the editor." **RUN** (they run — these are the README commands, on one line each):

```
$ unity status
$ unity command eval 'var g = UnityEngine.GameObject.CreatePrimitive(UnityEngine.PrimitiveType.Cube); g.name
= "Ground"; g.transform.position = new UnityEngine.Vector3(0, -0.25f, 0); g.transform.localScale = new
UnityEngine.Vector3(20, 0.5f, 20);'
$ unity command eval 'var p = UnityEngine.GameObject.CreatePrimitive(UnityEngine.PrimitiveType.Capsule);
p.name = "Player"; p.tag = "Player"; p.transform.position = new UnityEngine.Vector3(0, 1, 0); var rb =
p.AddComponent<UnityEngine.Rigidbody>(); rb.isKinematic = true; rb.useGravity = false;'
$ unity command eval 'return UnityEngine.GameObject.Find("Player") != null;'
# → True
```

LOOK FOR `eval 'return UnityEngine.GameObject.Find("Ground").transform.localScale;'` prints `(20.00, 0.50, 20.00)`; Hierarchy shows Ground and Player. The scene looks plainer than the next checkpoint (no walls, no materials) — that is expected; say so before someone asks. **PATROL** **eval takes statements, not expressions** — `eval 'x != null'` fails with "; expected"; it must be `eval 'return x != null;'`. That plus typos (missing semicolon, smart quotes from a chat app) are 90% of failures — have them copy from the README file, not from a message. `unity status` not ready = editor still compiling; wait for the spinner.

Hands-on Ex02 — the agent writes PlayerController, in plan mode (15 min)

Everyone opens `Ex02_Start.unity` (it already contains the polished Ex01 result). **SAY** "Open CLAUDE.md at the project root first — this is the contract every session reads. Hard rules: work only inside the exercise folder, never touch other ExNN folders, _Shared, _Setup, _Complete; never hand-edit scene YAML while the editor is connected; prove every change with eval or play mode." Then: **SAY** "Start `claude`, switch to plan mode, paste this:"

> Create a `PlayerController` MonoBehaviour in `Assets/Workshop/Ex02_Movement/Scripts/`. WASD/arrow keys move the player on the XZ plane (speed 6); holding the left mouse button (or a touch) moves the player toward the point under the

> cursor, raycast against the ground plane y=0. Clamp position to x and z in [-9, 9]. Rotate the player to face its movement direction. Then attach it to the "Player" object in the open scene using the Unity CLI, and tell me how you verified it compiles.

SAY while plans appear: "Read the plan before you approve. Two questions: which files will it create, and are they all inside Ex02_Movement? If it wants to touch anything else, say no and tell it to stay in the exercise folder." Have one volunteer read their plan aloud. **LOOK FOR** a single new file `Ex02_Movement/Scripts/PlayerController.cs`, attached via an eval like `AddComponent`, and the agent reporting a clean console. Verify:

```
$ unity command eval 'return UnityEngine.GameObject.Find("Player").GetComponent("PlayerController") !=
null;'
# → True   (on the Ex03+ checkpoints the script is namespaced:
GetComponent("Workshop.Ex03.PlayerController"))
$ unity command editor_play      # or press Play — WASD moves, click-to-move works, capsule turns toward the
click
```

PATROL Agents that "helpfully" create a new Player or a second Rigidbody — the kinematic Rigidbody on Player matters for Ex03's triggers; make them restore it. Agents that compile-check by reading the file instead of the console: ask "show me the console evidence". Anyone stuck at 10:45 opens `Ex03_Start.unity`, which contains working movement — that is the sync rule, not a failure.

Demo Ex03 — coin prefab and live inspection (7 min, you only)

Open `Ex03_Start.unity` on your machine. **RUN** in Claude Code:

> In `Assets/Workshop/Ex03_Coins/`: create a `CoinPickup` MonoBehaviour in `Scripts/` that spins the coin around world Y (120°/s) and, in OnTriggerEnter with the object tagged "Player", logs "Coin collected!" and destroys itself. Then build a Coin prefab in `Prefabs/`: root object with the CoinPickup and a trigger SphereCollider (radius 0.6), child cylinder visual (scale 0.9, 0.06, 0.9, rotated 90° on X) with its collider removed. Instantiate 8 coins in a circle of radius 6 at y=0.6 in the open scene, under a parent named "Coins". Use the Unity CLI to do the scene work.

Approve the plan; narrate what it does while it works (script, prefab, eight instantiations through eval). Then the line the whole afternoon rests on:

```
$ unity command eval 'return UnityEngine.GameObject.Find("Coins").transform.childCount;'
# → 8
# press Play, walk into one coin — console: "Coin collected!"
$ unity command eval 'return UnityEngine.GameObject.Find("Coins").transform.childCount;'
# → 7
```

SAY "I did not open an Inspector. I asked the running game a question and got a number. This is how we prove bugs this afternoon — not 'it looks fixed', but a number before and a number after."

Slides 6–9 [CliCommand] · security · maturity · up next (8 min)

SAY (slide 6) "Your existing editor tools become commands by adding one attribute — `[CliCommand]`. The agent then uses *your* validators and importers instead of inventing its own, and CI runs the same commands. This afternoon Track A writes a real one, `level_lint`." (slide 7) "Boundaries up front: agents write only to working branches; eval only in dev and in scope; builds under service accounts; everything in one audit log." (slide 8) walk the five production-readiness criteria — "nothing becomes mandatory on real projects until these are met." (slide 9) "After the break: three techniques for code you didn't write, then the full bug-to-proof loop live. This afternoon: engineering at 13:30, art at 15:15, one trainer, sit in on either."

PART 3 · 11:15 FOREIGN CODE + BUG-312 (DECK 03)

Slide 2 Technique 1 — code archaeology (10 min)

SAY "The agent is a superb guide and a confident liar. First prompt writes no code: it maps. Every claim must cite a path, and you verify two yourself." Show the payoff artifact on screen — open `Workshop/ARCHITECTURE.md` from the `ex02-start` branch of your demo clone (`git show ex02-start:Workshop/ARCHITECTURE.md`): scene flow Init → Main → `LevelList.LoadLevel`; the five classes (Board ~1,500 lines, GameManager, LevelData, Gem, UIHandler); match detection in `Board.DoCheck` (flood-fill then line extraction); boosters via `BonusGem`; levels as scenes with authoring tiles. **SAY** "Notice it's under a page and every section names a file. That is the bar."

Slide 3 Technique 2 — the contract (10 min)

SAY "The repo ships a stub CLAUDE.md with only hard safety rules. The real one is derived from the map." Show the canonical one (`git show ex03-start:CLAUDE.md`) and read three lines aloud: protected paths (Settings/, ProjectSettings/, YAML by hand); "Board.cs is the load-bearing wall — change it surgically"; the verification ritual (`recompile` + `recompile_status`, eval proof, root cause before code, `level_lint` must pass). **SAY** "Written down, every session runs the same ritual. Not written down, you babysit."

Slide 4 Technique 3 — git checkpoints (5 min)

RUN on your main clone: `./catchup.sh list` → the ten checkpoints. **SAY** "Every exercise has a start branch equal to the finished state of the previous one. Fall behind, run `./catchup.sh ex05`: your work is stashed, the branch is checked out, the

open editor is refreshed through the CLI. And the next branch is the previous exercise's solution — `git diff ex06-start..ex07-start` is the canonical fix for the bug I'm about to show you. Copying it is called reading the code review."

Demo BUG-312 — discovery to proven fix (40 min, on the ex06-start clone)

SAY read the report from slide 5: "Players burn through their moves too fast; the counter drops with nothing happening. One video shows a swap bouncing back — and the counter still went down. Started after the last gameplay patch." **Step 1 — reproduce by hand (3 min).** Play Level 1. Make a swap that does NOT create a match — the two gems animate back. Point at the HUD: the move counter went down anyway. **SAY** "The wrong fix is obvious and tempting: give players more moves. The right question is: who calls `LevelData.Moved()`, and when?" **Step 2 — investigate with proof demanded (15 min).** **RUN** in Claude Code, plan mode:

> Investigate BUG-312 [paste the report]. Find every call site of LevelData.Moved() and explain, from Board.cs's swap flow, exactly when a move should be consumed vs. when it currently is. Prove the root cause with evidence (code path + a play-mode eval showing RemainingMove dropping on a reverted swap) BEFORE proposing any change. Then apply the minimal fix and prove it the same way.

LOOK FOR the agent finding two call sites in `Board.TickSwap`: one in the matched branch (correct) and one in the revert branch — `Board.cs` around line 1140, under the comment "count the attempted move so players can't farm free retries". That comment is the seeded bug; the "last gameplay patch" in the report is exactly this. Narrate the agent's reading of TickSwap while it works. **Step 3 — the proof pair, on the projector (10 min).** Enter play mode on Level 1 (20 moves), then:

```
$ unity command --project-path . eval 'return "moves=" + Match3.LevelData.Instance.RemainingMove;'
# → moves=20
# make ONE failed swap by hand in the game view
$ unity command --project-path . eval 'return "moves=" + Match3.LevelData.Instance.RemainingMove;'
# → moves=19        ← BUG PROVEN: a reverted swap consumed a move
```

SAY "That is the evidence standard. Not 'I think', a number." Now let the agent apply the fix — it is a two-line deletion in the revert branch of `TickSwap` (the comment and the `LevelData.Instance.Moved();` call). If it proposes anything else — raising MaxMove, a cooldown, a flag — reject the plan in front of the room and say why. Recompile, play, repeat the pair:

```
$ unity command --project-path . recompile && unity command --project-path . recompile_status
# → no errors
# play mode, Level 1
# → moves=20 · failed swap · → moves=20      ← FIX PROVEN
# now a SUCCESSFUL swap                       → moves=19   (real moves still cost 1)
```

SAY "One line. Proven before, proven after, and the success path proven unchanged. This afternoon, Ex06, you do this exact loop at your own keyboard." **Fallback:** if the editor, the agent, or the network derails, play the recording without apologizing; the numbers are the same. Take five minutes of questions, then break.

PART 4 · 13:30 TRACK A — GEM HUNTER LAB (DECK 04)

13:30 Environment check (10 min, slides 1–2)

Walk the map on slide 2 (which exercises are hands-on, which are demos). Then everyone confirms, in this order:

```
$ unity status                          # ready
$ git branch -a | grep start            # ex01-start … ex09-start visible
$ git status                            # on main; a .mat and ProjectSettings.asset show as modified — the
editor touches them on open, catchup stashes them
$ ./catchup.sh ex01                     # everyone lands on the same commit
$ claude                                # opens; note the CLAUDE.md is a stub
```

SAY "The rule for the next ninety minutes: every exercise starts with everyone running `./catchup.sh exNN`. Finished, behind, absent — you all start identical. Nobody restores anything."

Ex01 Recon — map the codebase (13:40–13:55, hands-on) — 15 min

Your demo (3 min). **RUN** "Map this codebase: entry scene flow, where match detection lives, how boosters work, how levels are authored. Write nothing yet." Pick ONE claim from the answer — "match detection is in `Board.DoCheck`" — and open `Assets/GemHunterMatch/Scripts/Board.cs` on the projector to confirm it before believing it. **Their prompt.**

> Explore `Assets/GemHunterMatch/Scripts/` and produce `Workshop/ARCHITECTURE.md`: (1) the scene flow from Init to a playable level, (2) the five most important classes and one sentence each, (3) where match detection happens, (4) how a booster (BonusGem) plugs in, (5) how level data (moves, goals) is authored. Keep it under a page. Cite file paths for every claim.

SAY "When it's done, open two of the cited files yourself and check the claim is real. If a citation is wrong, tell the agent and make it fix the document — that is the exercise, not a failure." **LOOK FOR** the five classes named are Board, GameManager, LevelData, Gem, UIHandler; match detection attributed to `Board.DoCheck`; boosters to `BonusGem` with `CreateCustomMatch`; levels as scenes with LevelData + authoring tiles. Anything else is worth a public "is that true? open the file." **PATROL** Agents that

write code or "improve" something while mapping — stop them, the prompt says write nothing but the doc. Documents over a page with no paths — send back. Nobody verifying: ask a random person "which two claims did you check?"

Ex02 Contract — the real CLAUDE.md (13:55–14:10, hands-on) — 15 min

Everyone: `./catchup.sh ex02` (this brings the canonical ARCHITECTURE.md to anyone whose own was weak).
Your demo (3 min). Show the stub. Derive one rule live: SAY "ARCHITECTURE.md says level tuning lives in LevelData fields in the scene. So the contract says: *data changes go through LevelData and scene values, never hardcoded into Board.cs*. One observation, one rule." **Their prompt.**

> Using Workshop/ARCHITECTURE.md, rewrite CLAUDE.md as a real agent contract for this repo: protected areas (Settings/, ProjectSettings/, scene files by hand), where each kind of change belongs (gameplay logic / level data / UI / boosters), the Match3 namespace conventions, and the verification ritual (compile check via CLI, eval proof, play-mode test). Keep the existing hard safety rules.

SAY "Read it as if you had to obey it. Cut anything vague. Then the real test: open a FRESH claude session and ask 'what are you not allowed to touch here?' — it must answer from your file." LOOK FOR protected paths named; an address for boosters (subclass BonusGem, SmallBomb pattern), level data (LevelData + Board.ExistingGems in the scene), UI (UIHandler), editor tooling (Scripts/Editor, CliCommandAttribute); a verification ritual with recompile + eval + play mode; ideally "Board.cs is the load-bearing wall". The canonical version is `git show ex03-start:CLAUDE.md`. PATROL Contracts that are a wall of generic advice ("write clean code"). Ask: "point to a line that would have stopped a bad plan." The fresh-session test skipped — it is the only thing that proves the contract works.

Ex03–05 Level data · combo counter · Amethyst (14:10–14:30, your demos) — 3 × ~6 min

Everyone runs `./catchup.sh ex03` and watches; they will land on `ex06-start` afterwards with all three results in place. **Ex03 — LevelData.** SAY "Live-ops reality: 'make Level 2 harder' without a line of C#." Open `Scenes/Level2.unity`, show LevelData in the Inspector: MaxMove 20, first goal Count 6. RUN

> Open the Level2 scene with the Unity CLI and rebalance it: MaxMove from its current value to 4 fewer, and raise the first goal's Count by 5. Use the CLI's scene commands / eval — do not hand-edit the .unity file. Then save the scene and show me the new values as evidence.

LOOK FOR MaxMove 16, first goal Count 11 (that is the canonical `ex04-start` diff). Play Level 2: the HUD shows 16. Evidence command in play mode: `eval 'return Match3.LevelData.Instance.MaxMove;'` → 16. SAY "The agent used the editor through the CLI; the YAML never got opened by hand. That's the contract working." **Ex04 — ComboCounter.** Open `LevelData.cs`, show `OnGoalChanged` and `OnMoveHappened` (lines ~40–41). SAY "The hooks already exist. The skill is finding and using them instead of inventing an event bus." RUN the Ex04 prompt (ComboTracker in the Match3 namespace subscribing to both delegates, counting consecutive scoring moves, "Combo xN" in the HUD via the existing UI system, wired into the level scenes). When the plan appears ask the room: "is it reusing the delegates or inventing new plumbing?" — approve only the former. LOOK FOR a ~50-line `ComboTracker.cs`: increments on `OnGoalChanged` once per move window, resets in `OnMoveHappened` if the previous window scored nothing. Two scoring moves in a row → "Combo x2"; a dud move clears it. Evidence: `eval 'return UnityEngine.Object.FindFirstObjectByType<Match3.ComboTracker>().CurrentCombo;'`.
Ex05 — Amethyst. Show `Prefabs/GemWhite.prefab` (sprite, `GemType`, the `UISprite` field) and `Board.ExistingGems` on the Level1 scene's Board. RUN the Ex05 prompt (duplicate a gem prefab into "Amethyst", purple tint via SpriteRenderer color, next unused GemType, register in Level1's Board.ExistingGems via CLI, modify no C#). LOOK FOR `GemAmethyst.prefab` with GemType 6 and a color around (0.62, 0.35, 0.95); `eval 'return UnityEngine.Object.FindFirstObjectByType<Match3.Board>().ExistingGems.Length;'` went up by one; purple gems fall and match in play mode. SAY "Zero C# changed. Track B reskins this gem too this afternoon — it exists because the system is data-driven." Close the block: everyone `./catchup.sh ex06`.

Ex06 BUG-312 — the root-cause hunt (14:30–14:45, hands-on) — 15 min

SAY "This morning's loop, your keyboard, the same standard. The scene you just synced to carries the bug. Rule: no fix until the root cause is proven — and I will reject symptom patches personally." **Their prompt** — the Ex06 file's prompt, with the report pasted (see Part 3). **Their evidence**, in play mode:

```
$ unity command --project-path . eval 'return "moves=" + Match3.LevelData.Instance.RemainingMove;'
# 20 → failed swap → 19 (bugged)      after the fix: 20 → failed swap → 20, successful swap → 19
```

LOOK FOR the agent's explanation naming the `Moved()` call in `TickSwap`'s revert branch (Board.cs ~1140) and the fix being that deletion only. Their before/after numbers on screen — this is a closing deliverable. PATROL Walk the room continuously. Plans that raise MaxMove, add a "free retry" flag, clamp the counter, or debounce swaps: reject aloud — "that hides the symptom; where is the call site?" Agents that fix first and prove after: make them run the buggy eval pair first anyway (they can `git stash`, prove, `git stash pop`). Anyone lost at 14:42: `git diff ex06-start..ex07-start` and read it with them.

Ex07 level_lint — their own CLI command (14:45–14:55, hands-on) — 10 min

Everyone `./catchup.sh ex07`. **Hook (1 min):** RUN `unity command --project-path . | grep lint` → nothing.
SAY "Let's change that. Same attribute you saw this morning, now guarding a real content pipeline." **Their prompt.**

> Create `Assets/GemHunterMatch/Scripts/Editor/LevelLint.cs`: an editor class registering a CLI command `level_lint` (CliCommandAttribute from the com.unity.pipeline package) that validates the OPEN scene: (1) a LevelData exists with MaxMove > 0, (2) every Goal has Count > 0 and a non-null Gem, (3) every Goal's gem type exists in Board.ExistingGems, (4) the board has at least one spawner (Board.SpawnerPosition non-empty in play mode — outside play mode check it via the scene's authoring tiles if simpler). Output a readable PASS/FAIL report; fail loudly per broken check.

LOOK FOR a static method with `[CliCommand("level_lint", "…")]` in namespace `Unity.Pipeline.Commands`, returning a string report with one line per check. Then the sabotage test, which is the actual verification:

```
$ unity command --project-path . level_lint                    # PASS — level is valid, [ok] per check
$ unity command --project-path . eval 'Match3.LevelData.Instance.Goals[0].Count = 0; return "sabotaged";'
# in play mode
$ unity command --project-path . level_lint                    # FAIL — names the broken check
$ unity command --project-path . eval 'Match3.LevelData.Instance.Goals[0].Count = 6; return "restored";'
```

PATROL Linters that only print PASS (never fail) — the sabotage test exposes them. Command not appearing in the list = the class is not in an Editor folder or didn't compile: `recompile_status`. Reference: `git show ex08-start:Assets/GemHunterMatch/Scripts/Editor/LevelLint.cs`.

Ex08 + Ex09 Cross Bomb briefing · iOS build kicked off (14:55–15:00)

5 min

Ex08 briefing (2 min). Read FEAT-88 from the exercise file aloud: a bonus gem that clears its entire row AND column, following the BonusGem pattern, a prefab with a distinct look, a trigger sound reusing an existing clip; must not break normal matches, other boosters, move counting, or level_lint. Open `Scripts/BonusGem/SmallBomb.cs` and walk the contract once: `m_Usable = true` in Awake; `Use()` builds a forced-deletion match via `Board.CreateCustomMatch(m_CurrentIndex)`; each affected cell goes through `HandleContent`. **SAY** "No prompt template for this one. After the workshop: your prompt, your plan review, your play-mode proof that the full row and column cleared, and level_lint still green. The canonical answer is `git diff ex08-start..ex09-start` — but write yours first." **Ex09 build (3 min).** `./catchup.sh ex09` on your machine. Show `BuildLevelList : IPreprocessBuildWithReport` in `LevelList.cs`: **SAY** "At build time this fills Build Settings with the level scenes. Reading other people's build plumbing is the exercise." Kick it:

```
$ unity command --project-path . build --target iOS --outputPath Builds/iOS --confirm true
$ unity command --project-path . build_status
```

LOOK FOR on a fresh clone the build goes straight through — Build Settings already match the LevelList asset. To show the preprocessor doing its job, make it drift first: `unity command --project-path . eval 'var k = new System.Collections.Generic.List<UnityEditor.EditorBuildSettingsScene>(); foreach (var s in UnityEditor.EditorBuildSettings.scenes) if (!s.path.Contains("/Level")) k.Add(s); UnityEditor.EditorBuildSettings.scenes = k.ToArray(); UnityEditor.AssetDatabase.SaveAssets(); return k.Count;'` (the level scenes are gone from Build Settings), then build: it stops once with "Level List had to be rebuilt, restart the build" and a "Build Stopped" warning, the level scenes are back, and the restart succeeds. **SAY** "The preprocessor fixed Build Settings and refused to build on a list it just changed — read it and tell me why it stops rather than continuing." The report will show ~13 "More than one global light" errors: they come from the pipeline package opening every scene additively while it scans for RuntimePipelineManager components, they are non-fatal, and the result still reads Succeeded — a good CI conversation for the closing. The first iOS export takes several minutes; repeat builds take ~20 seconds. An Xcode signing failure at the very end is fine: the pipeline is the lesson, not the provisioning profile. **The CI contrast (1 min, don't skip).** What you just ran is the editor half of the CLI: `unity command build` asked the *open* editor to build. The CI half is the same build with no editor window: **RUN** or just show `unity build . --target iOS --allow-install` on a closed project — the CLI launches its own headless editor in batch mode and returns an exit code. **SAY** "Same preprocessor, same scenes, same BuildReport. Two things change: nobody is logged in, so it needs a service account and a license seat (`unity auth login --client-id … --secret-from-stdin`, `unity license activate`), and nothing interactive may run — which is exactly why that modal dialog had to go from the preprocessor. Everything else you did today — eval, level_lint, catchup refreshing the editor — is editor-half only. It does not exist in a Player build; the pipeline package even tells you so in the build warnings." Closing slide 4 picks this up: level_lint runs in CI as a gate, not as a live command.

PART 5 · 15:15 TRACK B — THE RESKIN (DECK 05)

Slides 1–2, 5 Talk (10 min)

SAY "Gem Hunter is Unity's 2D lighting and VFX showcase, so your reskin isn't just sprites — it lives with point lights, normal maps and match effects. The surface is clear: six gem sprites plus the Amethyst engineering added an hour ago, the HUD goal icons, and the background. The approval checklist and provenance rules from this morning apply unchanged." Then slide 5's technical notes — these are the four things that go wrong: PPU and pivot must match the sample; change the prefab, not the scene; the Gem component's `UISprite` field feeds the HUD; provenance in SOURCES format.

Flow 1 Generate » clean up » import » swap (15:25–16:00)

35 min

Step 1 — generate as a set (10 min). **SAY** "Seven gems, one family. One prompt template in the studio style model; the only variable is the color word. Three variations each." Reference gems and their sprites are in `Assets/GemHunterMatch/Images/Gameplay pieces/` — `Sprite_BlueItem.png`, Green, Purple, Red, White (+ the shipped sixth

and the Amethyst). Output: square, transparent background, 1024×1024. Every prompt goes into their SOURCES record as they go, not afterwards. **Step 2 — clean up and normalize (5 min).** Background removed, cropped to the same bounding box, same visual weight; export one PNG per gem named to match the originals (`Gem_Blue.png` …). Anyone without Scenario access gets your fallback set now and continues. **Step 3 — copy the import standard (5 min).** Open a sample sprite's import settings and read them aloud; theirs must match exactly:

Setting	Sample value	Why it matters
Texture Type / Sprite Mode	Sprite (2D and UI) / Single	Anything else won't slot into SpriteRenderer
Pixels Per Unit	**512**	The classic failure: a giant or tiny gem on the board
Pivot	Center (0.5, 0.5)	Board alignment; gems bounce around their pivot
Filter Mode	Bilinear	Matches the rest of the board
Max Size	2048	Keep the sample's budget

SAY "The sample gems also ship a `_n` normal map and a `_mask`. Today you replace the base sprite only; the lights will still shade the shape, just flatter. Normal maps for your set are a follow-up task, not a today task." **Step 4 — swap on the prefab (15 min).** SAY "The change goes on the gem prefabs, so all three levels update at once. Two fields: the SpriteRenderer's sprite AND the Gem component's UISprite — miss the second and the HUD icons stay old." Note the originals use a *separate* icon set for the HUD (`Images/UI/Gems/SP_Gem_*.png`) — so each gem needs a board sprite and a HUD icon; reusing the board sprite for both is acceptable today. Their prompt to the agent:

> For each gem prefab in `Assets/GemHunterMatch/Prefabs/` (GemBlue, GemGreen, GemPurple, GemRed, GemWhite, GemYellow, GemAmethyst), set the SpriteRenderer sprite to the matching new board sprite under `Assets/Reskin/Board/` and the Gem component's UISprite to the matching new HUD icon under `Assets/Reskin/UI/`. Do it on the prefab assets through the Unity CLI (eval + AssetDatabase), not by editing YAML, and not in the scene. Save assets and show me each prefab's sprite name as evidence.

LOOK FOR evals of this shape per prefab (the agent will write them; you recognize them): load the prefab and the sprite with `AssetDatabase.LoadAssetAtPath`, set `GetComponent<SpriteRenderer>().sprite` and `GetComponent<Match3.Gem>().UISprite`, `EditorUtility.SetDirty`, `AssetDatabase.SaveAssets`. Then Level 1 in play mode: new gems on the board, new icons in the goal HUD. PATROL Gems huge or tiny → PPU isn't 512. Board misaligned → pivot. HUD icons unchanged → UISprite not set. Someone editing the scene instance instead of the prefab → "does Level 2 show it too?"

Flow 2 Light » VFX » verify » approve (16:00–16:40) — 40 min

Step 5 — 2D lights (10 min). In the Level 1 scene: the global Light2D and the point lights. Tune color and intensity to the new palette. The test: SAY "Seven gems on the dark board — can you tell every one apart at a glance? If two blur together, raise value contrast on the sprite, not the light." **Step 6 — match VFX (10 min).** Reality check first: every gem's match effect is the shared `VFX/BasicEffects/VFX_CellCrash` prefab, and its graph exposes *no* parameters — there is no color knob to turn. The per-gem hook is the Gem component's `MatchEffectPrefabs` array: a gem can point at its own effect prefab. SAY "Today the match effect stays as it is — it reads fine on any palette. If your set clashes with it, the follow-up is a duplicated VFX_CellCrash per color assigned through MatchEffectPrefabs, not editing the shared graph." Use the 10 minutes on lighting instead. **Step 7 — verify in game (5 min).** A full Level 1 run: gems read, HUD goal icons show the new set, the shared match effect reads well on the new palette, no console errors. Before/after screenshots from the same camera position. **Step 8 — approval package (15 min).** Checklist from the guide's appendix filled in, SOURCES record (model, prompt, version per gem), before/after. Then the cross-approval rehearsal: pairs swap packages and each writes an approve/reject decision with reasoning. SAY "A rejection with a reason is worth more to the prompt library than an approval." LOOK FOR every package has all three parts and a written decision on it. PATROL Provenance written from memory at the end instead of during generation. "Approved" with no reasoning. Anyone still generating at 16:15 — freeze the set and move on; consistency beats polish.

PART 6 · 16:45 CLOSING (DECK 06)

Slides 1–2 Show & tell and the merge (20 min)

Track A (7 min). One volunteer shows the BUG-312 proof pair on the projector (their before/after numbers) and runs their `level_lint` through a sabotage. You show `build_status` from the Ex09 build. **Track B (7 min).** One before/after reskin, Level 1 in play mode, HUD included. Ask the cross-approver to read their decision. **The merge (6 min).** On your machine: the reskin assets (Prefabs + Reskin sprites) copied onto your `ex09` working state, one commit, then `unity command --project-path . build --target iOS --outputPath Builds/iOS-Final --confirm true`. SAY "The game nobody here wrote this morning belongs to both of you tonight." Let it build while you talk.

Slides 3–4 The four traps and the next 90 days (20 min)

SAY (slide 3) with today's examples: confident misattribution — "who caught a wrong citation in Ex01?"; invented plumbing — "the ComboTracker plan that wanted an event bus"; symptom patching — "every MaxMove plan I rejected"; contract-less

sessions — "the fresh-session test in Ex02." (slide 4) the concrete steps: checkpoint discipline on real projects (feature branches + a proof section in every PR); level_lint → production lint in CI; an ARCHITECTURE.md + CLAUDE.md pair per active repo, owned by the champions; metrics quarterly — **SAY** "put the date of the next review on the calendar before you leave this room."

Slide 5 Collect and close (5 min)

Collect the three metrics templates — do not let them leave empty. Confirm the materials repo link is shared. Thanks.

WHEN THINGS BREAK

Symptom	Your move
`unity status` not "ready"	Editor compiling or importing — wait for the spinner. Persists: `unity pipeline list`; check the editor isn't in Safe Mode (a compile error on open).
`eval` returns an error	It is a C# compile error in the snippet: a smart quote, a missing `UnityEngine.` prefix, a generic written as <T> inside single quotes. Copy from the README file, not chat.
Agent plan reaches outside its lane	Reject in front of everyone and read the CLAUDE.md line it violates — the best teaching moment of the day.
Participant hopelessly behind (Track A)	`./catchup.sh exNN` — work is stashed (`git stash list` to recover), branch checked out, editor refreshed. Identical to the room in ten seconds.
Someone wants the solution	`git diff exNN-start..exNN+1-start` — read it with them; the next branch is the canonical answer.
Editor confused after a branch switch	catchup.sh already ran `AssetDatabase.Refresh`; File > Open Recent to reopen the scene. If scripts don't recompile: `unity command --project-path . recompile`.
Coin Rush scene broken in the morning	Close without saving, reopen the current Start scene. Worst case: `Workshop > Build All Checkpoints` or `unity command eval 'Workshop.Setup.WorkshopBuilder.BuildAll();'` regenerates every checkpoint.
BUG-312 demo derails	Play the recording without apology; the numbers are identical. Never debug live for more than two minutes in front of the room.
Fix leaked into your main clone	That's why the demo runs on the ex06-start clone. If it happened anyway: `git checkout -- Assets/GemHunterMatch/Scripts/Board.cs` on main.
Scenario down / no access	Hand out the pre-generated set from your machine; Track B continues from Step 2.
Gems giant / tiny / misaligned	Import settings vs. a sample sprite: PPU 512, pivot center. It is always one of those two.
HUD icons still old after the swap	The Gem component's `UISprite` field was not set — SpriteRenderer alone is not enough.
Want a HUD screenshot as evidence	`unity command capture_game_view --source screen --save_path Workshop/hud.png` in play mode — the path must be inside the project root.
Build stops with "Level List had to be rebuilt"	The preprocessor changed Build Settings (a level scene was missing). Restart once — the second run goes through.
Build report lists 13 "More than one global light" errors	Pipeline-package noise from scanning scenes additively; the result is still Succeeded. Ignore.
`build_status` seems stuck on "building"	The editor's main thread is busy; the call can block for the length of the build. Poll every 10–15 s and be patient on the first export.

APPENDIX · COMMAND CRIB SHEET

```
# editor half — needs a running editor with com.unity.pipeline
unity status                                        connected editor and its state
unity command [--project-path .]                    list exposed commands (path flag needed in Gem Hunter)
unity command eval '<C#>'                           run C# live; 'return x;' to get a value back
unity command --project-path . recompile            compile scripts · then recompile_status for errors
unity command --project-path . console              read the editor console from the terminal
unity command editor_play                           enter play mode from the terminal
unity command --project-path . level_lint           (after Ex07) validate the open level
unity command --project-path . build --target iOS --outputPath Builds/iOS --confirm true
unity command --project-path . build_status         poll the async build
# CI half — no editor open; starts its own headless editor
unity build . --target iOS --allow-install          headless Player build, exit code out (CI: service acct +
seat)
unity test . --mode EditMode --report-format junit  headless test run for CI
./catchup.sh list | ex01..ex09 | complete           checkpoints; stashes your work, refreshes the editor
git diff exNN-start..exNN+1-start                   the canonical solution of exercise NN
git show ex02-start:Workshop/ARCHITECTURE.md        canonical recon · ex03-start:CLAUDE.md canonical contract
eval 'return "moves=" + Match3.LevelData.Instance.RemainingMove;'        BUG-312 proof
eval 'return Match3.LevelData.Instance.MaxMove;'                          Ex03 proof (play mode)
eval 'return FindFirstObjectByType<Match3.Board>().ExistingGems.Length;'   Ex05 proof (UnityEngine.Object.)
eval 'return FindFirstObjectByType<Match3.ComboTracker>().CurrentCombo;'   Ex04 proof (UnityEngine.Object.)
eval 'return UnityEngine.GameObject.Find("Coins").transform.childCount;'   Coin Rush live count
```